

Test Pattern

Disciplina: Testes de Software

Aluna: Érica Alves dos Santos

Matrícula: 799648

1. Padrões de Criação de Dados (Builders)

1.1 Por que CarrinhoBuilder e não CarrinhoMother?

O padrão **Object Mother** cria instâncias fixas e nomeadas de entidades simples. Ele é ideal para objetos cujos atributos variam pouco entre cenários, como tipos de usuário (**PADRAO** ou **PREMIUM**), onde dois métodos estáticos já cobrem todos os casos de teste relevantes.

O **Carrinho**, contudo, é uma entidade composta: pode ter diferentes usuários, listas de itens variadas, ou estar completamente vazio. Aplicar Object Mother aqui levaria a uma explosão de métodos:

`umCarrinhoVazio()`, `umCarrinhoComUmItem()`, `umCarrinhoComDoisItens()`, `umCarrinhoDeUsuarioPremiumComTresItens()`, tornando a classe impossível de manter.

O padrão **Data Builder** resolve esse problema com uma API fluente: o construtor define valores padrão sensatos, e os métodos encadeáveis (`.comUser()`, `.comItens()`, `.vazio()`) permitem customizar apenas o que é relevante para cada cenário, mantendo o código de setup enxuto e expressivo.

1.2 Exemplo Antes × Depois

ANTES — Setup manual (Test Smell: Obscure Setup)

```
// Setup manual – tudo exposto, nada evidente
const user = new User(2, 'Maria Premium', 'premium@email.com',
  'PREMIUM');
const item = new Item('Notebook', 200);
const carrinho = new Carrinho(user, [item]);

// O leitor precisa deduzir o que importa para o teste...
```

DEPOIS — Setup com Data Builder

```
// Intenção explícita: usuário premium, R$ 200 em itens
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([new Item('Notebook', 200)])
  .build();
```

1.3 Como o Builder melhora legibilidade e manutenção

Legibilidade: cada teste declara explicitamente apenas os atributos relevantes para aquele cenário. Um leitor entende imediatamente que o foco do teste é o desconto premium, não precisa dissecar a construção do objeto.

Manutenção: se a assinatura do construtor de `Carrinho` mudar (por exemplo, adicionando um campo `cupomDesconto`), basta atualizar o valor padrão no Builder. Sem o Builder, cada teste que constrói um Carrinho manualmente precisaria ser alterado.

Prevenção de Test Smells: o padrão elimina diretamente o smell *Obscure Setup*, pois oculta a complexidade da construção e expõe somente a variação intencional de cada teste.

2. Padrões de Test Doubles (Mocks vs. Stubs)

Test Doubles são substitutos de dependências externas usados em testes de unidade. Eles permitem testar a lógica de negócio de forma isolada, sem acionar serviços reais como gateways de pagamento ou servidores de e-mail.

Tipo	Propósito	O que verifica
Dummy	Preenche parâmetro obrigatório	Nada (não deve ser chamado)
Stub	Controla valor de retorno	Estado (resultado do método)
Mock	Registra chamadas para verificação	Comportamento (interações)

2.1 Cenário analisado: cliente Premium finaliza a compra

Neste cenário um usuário PREMIUM compra R\$ 200,00 em itens. Espera-se que o sistema aplique 10% de desconto (cobrando R\$ 180,00), persista o pedido e envie um e-mail de confirmação.

```
// — Arrange —
const usuarioPremium = UserMother.umUsuarioPremium();
const carrinho = new CarrinhoBuilder()
  .comUser(usuarioPremium)
  .comItens([new Item('Notebook', 200)])
  .build();

// STUB — controla o retorno; não nos importa como foi chamado
const gatewayStub = { cobrar: jest.fn().mockResolvedValue({ success: true }) };
const repositoryStub = { salvar: jest.fn().mockResolvedValue(pedidoSalvoEsperado) };

// MOCK — registra chamadas para verificação posterior
const emailMock = { enviarEmail: jest.fn().mockResolvedValue(undefined) };
};
```

```
// — Act —————
const pedidoRetornado = await checkoutService.processarPedido(carrinho,
cartao);

// — Assert – Verificação de Comportamento —————
expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, cartao);
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  'premium@email.com', 'Seu Pedido foi Aprovado!',
  expect.stringContaining('99'));

```

2.2 Por que GatewayPagamento é (principalmente) um Stub?

O papel central do GatewayPagamento no fluxo é **retornar um resultado** (`{ success: true/false }`) que direciona o caminho da lógica de negócio. O que o teste precisa controlar é esse retorno — não verificar se o gateway foi invocado.

Isso caracteriza **Verificação de Estado**: o teste afirma que, dado um retorno `success: false`, o estado do resultado (`pedido`) é `null`. O Stub fornece o valor controlado sem impor expectativas de chamada.

Observação: mesmo usando um Stub para o gateway, ainda verificamos o argumento passado a `cobrar()` (180 em vez de 200) para validar a regra de desconto. Essa verificação pontual de argumento é uma escolha de design: o gateway continua sendo essencialmente um Stub, mas também participa de uma asserção específica de comportamento.

2.3 Por que EmailService é um Mock?

O envio de e-mail é um **efeito colateral** do checkout bem-sucedido: o método não retorna nenhum valor utilizável pela lógica de negócio. O único modo de verificar que ele funcionou corretamente é observar *se foi chamado, quantas vezes e com quais argumentos*.

Isso caracteriza **Verificação de Comportamento**: o teste afirma que `enviarEmail` foi acionado exatamente uma vez, com o endereço correto e o assunto esperado. Sem um Mock que registre essas interações, seria impossível garantir que o e-mail foi enviado, ou que não foi enviado duas vezes.

Critério	GatewayPagamento (Stub)	EmailService (Mock)
Objetivo principal	Controlar o retorno do pagamento	Verificar que o e-mail foi enviado
Verificação	Estado (pedido retornado)	Comportamento (chamadas)
Método Jest	<code>mockResolvedValue({ success: true })</code>	<code>toHaveBeenCalledWith(...)</code>
Falha do teste se...	Retorno não controla o fluxo	E-mail não for enviado / errado

3. Conclusão

Este trabalho demonstrou como Padrões de Teste funcionam como uma estratégia proativa de qualidade: em vez de detectar e corrigir Test Smells depois que surgem, os padrões os previnem desde o início da escrita dos testes.

O **Data Builder** elimina o smell *Obscure Setup* ao centralizar a construção de objetos complexos e expor apenas as variações relevantes para cada cenário. A consequência prática é imediata: testes mais curtos, mais legíveis e muito mais fáceis de atualizar quando o domínio evolui. O **Object Mother**, por sua vez, cobre entidades simples e estáveis com métodos nomeados que comunicam intenção — `umUsuarioPremium()` diz mais do que `new User(2, 'Maria', 'email', 'PREMIUM')`.

A distinção entre **Stubs** e **Mocks** revelou-se fundamental para escrever testes focados. Stubs controlam o fluxo de dependências externas e permitem *Verificação de Estado* — ideal para dependências cujo retorno direciona a lógica de negócio. Mocks registram interações e habilitam *Verificação de Comportamento* — indispensável para efeitos colaterais como notificações, que não retornam valores testáveis.

Juntos, esses padrões produzem uma suíte de testes **sustentável**: cada teste é independente, compreensível em isolamento e resiliente a mudanças de implementação que não alteram o comportamento externo. Isso traduz diretamente em confiança para refatorar, evoluir e entregar software de qualidade.

Resumo dos padrões aplicados:

- **UserMother** (Object Mother) — usuários fixos
- **CarrinhoBuilder** (Data Builder) — setup flexível
- Stub em **GatewayPagamento** e **PedidoRepository** — controle de fluxo
- Mock em **EmailService** — verificação de efeito colateral
- Padrão AAA (Arrange–Act–Assert) em todos os testes